

SOFTVER PROJEKTA

SEKCIJA ROBOTIKE STŠ SOMBOR

Povezivanje robotske i ljudske šake

ABSTRACT. U ovom radu opisan je softver sistema α i sistema ω .

SISTEM α

KALIBRACIJA KAMERA

Kalibracija kamere jeste postupak kojim se utvrđuje matrica kamere $\mathbf{P}$, takva da je

$$\mathbf{P}\mathbf{X} = \alpha\mathbf{x}, \quad (1)$$

gde je $\mathbf{X}$ proizvoljna tačka prostora unutar vidnog polja kamere, $\mathbf{x}$ položaj iste tačke na slici, a $\alpha \in \mathbb{R}$. Kalibracija kamere omogućava triangulaciju tačke. Kalibracija kamere radi se jedanput, nakon što se kamera fiksira — nakon toga, matrica se više ne menja (zavisí od položaja kamere te fizičkih osobina sočiva i senzora) i nema potrebe ponavljati kalibraciju.

Kalibracija kamere obavlja se pomoću funkcije *calibrateCamera* biblioteke *cv2* (OpenCV), na osnovu test-slika (iste) šahovnice. Funkcija prima:

- (1) niz nizova vektora $((\mathbf{S}))$, koji daje koordinate tačaka mreže šahovnice u njenom sopstvenom koordinatnom sistemu, izražene u jedinicama stranice polja šaho-vnice. Dat je jednakošću (2), gde je w širina, a h visina šahovnice izražena u istim jedinicama. Svi nizovi $(\mathbf{S})$ međusobno su jednaki — njihovo umnožavanje stvar je implementacije funkcije.

$$(\mathbf{S}) = (a_n)_{0 \leq n \leq w}^{d=1} \times (b_n)_{0 \leq n \leq h}^{d=1} \quad (2)$$

- (2) niz nizova vektora $((\mathbf{x}))$, koji daje za svaku sliku koordinate tačaka mreže šahovnice u pikselima u koordinatnom sistemu kamere. Niz $(\mathbf{x})$ za datu (sivu) sliku utvrđuje se evaluacijom funkcije *findChessboardCorners* biblioteke *cv2* nad tom slikom, poznajući dimenzije šahovnice u jedinicama stranice polja.
- (3) vektor $\mathbf{d}$, koji daje dimenzije slike u pikselima. U našem slučaju, to je vektor $\begin{bmatrix} 280 \\ 320 \end{bmatrix}$, jer koristimo slike širine 320px i visine 280px.

Izlaz f-je *calibrateCamera* jeste 5-torka, čiji je najvažniji element matrica **K**, takva da je:

$$\mathbf{P} = \mathbf{K}[\mathbf{R}|\mathbf{t}] \quad (3)$$

Matrica **R** jeste matrica rotacije kamere, a vektor **t** — vektor translacije kamere.

Ispod je priložen kod. U njemu nije obavljena obrada matrice **K** — to se radi u rutini za trijagulaciju. Napomena: konvencija imenovanja je izmenjena!

```
import cv2
from matplotlib import pyplot as plt
import numpy as np
import glob

criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

objp = np.zeros((7*9,3), np.float32)
objp[:, :2] = np.mgrid[0:9,0:7].T.reshape(-1,2)

objpoints = []
imgpoints = []

images = glob.glob('/home/veljaaas/Documents/sekcija/kamereDruga/*.png')
images.sort()
print(images)

for fname in images:
    print(fname)
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY)

    ret, corners = cv2.findChessboardCorners(gray, (9,7), None)

    if ret == True:
        objpoints.append(objp)

        corners2 = cv2.cornerSubPix(gray,corners, (11,11), (-1,-1), criteria)
        imgpoints.append(corners2)

        cv2.drawChessboardCorners(img, (9,7), corners2, ret)
        plt.imshow(img)
        plt.show()

ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpoints,
    imgpoints,
    gray.shape[::-1],
    None,
    None
)
```

PRIKUPLJANJE SLIKA

Slike s kamera stalno preuzimaju dve zasebne niti, kako se ne bi zapunio bafer te kako bi glavna nit mogla dobijati slike u realnom vremenu (tj. kako glavna nit ne bi unosila kašnjenje u čitanje slika). Beleže se samo najnoviji kadrovi s obeju kamera. Slike se preuzimaju metodima *grab* i *release* klase *VideoCapture* biblioteke *cv2*. Metodom *grab* prihvata se kadar iz bafera u privremeno skladište, pa se metodom *release* isti predaje programu. Primer koda za jednu kameru priložen je ispod.

```
def citajKamera1():
    global kamera1Najnoviji
    global camera1
    while True:
        if camera1.grab():
            ret, kamera1Najnoviji = camera1.retrieve()
nit1 = threading.Thread(target = citajKamera1, args=())
nit1.start()
```

OBRADA SLIKE

Cilj obrade slike jeste merenje koordinata svih vrhova prstiju i drugih značajnih tačaka šake na slikama s obeju kamera. Implementirana su dva metoda obrade slike: primenom biblioteke *mediapipe* (pristup koji smo pozajmili) i primenom segmentacije po boji (pristup koji smo osmislili).

- (1) Biblioteka *mediapipe* nudi istrenirani VI model za detekciju 21 značajne tačke dlana. Priloženim isečkom koda može se formirati niz vektora kojim su date koordinate značajnih tačaka dlana na slici, ukoliko se dlan vidi dobro na slici.

```
try:
    detection_result = detector.detect(image)
    koordinateA = np.array([(a.x*320, a.y*240)
                           for a in detection_result.hand_landmarks[0]])
except:
    continue
```

- (2) Osnovna ideja postupka sa segmentacijom po boji jeste da, ukoliko ofarbamo svaki prst u drugu boju (kao i „koren” dlana — referentnu tačku), sve ostalo u kadru učinimo hromatski neutralnim, uzmemo sliku te odredimo centre masa pojedinačnih boja, možemo doznati koordinate svih prstiju na toj slici.

Uzimaju se dve slike (iz razloga koji će kasnije biti definisan) — jedna otvorene šake, i jedna šake u željenom položaju, tako da se između njih nije desila rotacija šake (translacija je dopuštena i ne utiče na dalje merenje). Nad obema slikama primenjuje se Gausov NF filter u cilju smanjivanja detaljnosti slike (relevantni su samo veliki klasteri boja na vrhovima prstiju; detekcija drugih, neočekivanih stvari narušava rad programa). To omogućava f-ja *GaussianBlur* biblioteke *cv2*.

Slike se potom prevode u $L * a * b$ prostor boja, odabran iz tog razloga što je u njemu nijansa svetla (koja će ubrzo biti definisana) poprilično invarijantna po intenzitetu svetlosti (ista površina uvek će imati približno istu nijansu pod svetlom istog frekv. spektra). U odabranom prostoru boja, boja je zadata vektorom $[L \ a \ b]^T \in (\mathbb{R}^3)^T$, $a^2 + b^2 \in [0, c_0^2]$, $L \in [0, L_0]$. Parametar L odnosi se na osvetljaj, a a i b — na hromatski sadržaj. Nijansu možemo definisati kompleksnim brojem:

$$\hat{c} = a + jb \quad (4)$$

(projekcija boje na ab ravan). Moduo tog broja $c = \|\hat{c}\| = \sqrt{\hat{c}\hat{c}^*} = \sqrt{a^2 + b^2}$ govori o zasićenosti boje, a argument $\phi = \arg \hat{c} = \arctg \frac{b}{a}$ — o tome je li boja crvena, zelena itd. ($\phi = \pi \pm \pi$ — crvena, $\phi = \pi/2$ — žuta...). Baratajući isključivo tim brojem (stavljanjem c i ϕ u granice) mogu se definisati pragovi detekcije prstiju. Na osnovu tih pragova mogu se generisati binarne slike koje označavaju pojedinačne prste. Primer implementacije dat je ispod (za plavu ref. tačku). Napomena: konvencija imenovanja je izmenjena!

```
slika1cir = np.array(slika1fi.copy(), dtype=np.int32)
maska1ir = 255*(np.arctan2(slika1cir[:, :, 2]-128,
    slika1cir[:, :, 1]-128) > -np.pi/2)
    & (np.arctan2(slika1cir[:, :, 2]-128,
    slika1cir[:, :, 1]-128) <= -np.pi/3.5)
    & (np.abs(slika1cir[:, :, 1]-128
    + 1j*(slika1cir[:, :, 2]-128)) > 0.1*256))
```

Kad su izdvojene binarne slike, koordinate centra mase lako se određuju primenom funkcija *argwhere* i *mean* biblioteke *numpy*. Funkcija *argwhere* uzima n -dimenzionalni niz i vraća niz 2-vektora ($\mathbf{x}$) gde svako $\mathbf{x}$ daje koordinate jedne od tačaka u kojima je zapamćena vrednost različita od 0. Funkcija *mean* vraća srednju vrednost prosleđenog niza. Isečak koda ispod malo je izmenjen radi lakšeg razumevanja imenovanja.

```
def centarMase(maska1):
    kz1 = np.argwhere(maska1)
    cxz1 = np.mean(kz1[:, 1])
    cyz1 = np.mean(kz1[:, 0])
    return (cxz1, cyz1)
```

Saznavanjem centara mase binarnih slika završena je obrada slike. Mana ovog pristupa jeste osetljivost na uslove osvetljenosti (iako je osetljivost na intenzitet svetlosti sprečena). Ne mogu se tako lako odabrati dovoljno veliki segmenti spektra boja kao granice za svaki prst jer bi se preklapili!

TRIANGULACIJA TAČAKA

Triangulacija tačke jeste problem utvrđivanja položaja tačke u prostoru na osnovu njenog položaja u dvema slikama, vremenski koincidentalnim a prostorno nekoincidentalnim. Ukoliko poznajemo položaje tačke na slikama s kamera 1 i 2 $\mathbf{r}_1$ i $\mathbf{r}_2$ te matrice kamera 1 i 2 $\mathbf{P}_1$ i $\mathbf{P}_2$ problem se svodi na nalaženje takvih realnih parametara α i β i takve tačke $\mathbf{R}$ da bude

$$\mathbf{r}_1 = \alpha \mathbf{P}_1 \mathbf{R} \bigwedge \mathbf{r}_2 = \beta \mathbf{P}_2 \mathbf{R}. \quad (5)$$

Taj sistem jednakosti, međutim, u ovom obliku nije naročito koristan, budući da ima tri nepoznate. Primitimo da važi

$$\mathbf{P}_i \mathbf{R} \times \mathbf{r}_i = \mathbf{0}. \quad (6)$$

Tako su iz posla sklonjeni parametri α i β , i jedina preostala nepoznata jeste $\mathbf{R}$. Uz određene transformacije:

$$\begin{bmatrix} y_1 \mathbf{p}_{13}^\dagger - \mathbf{p}_{12}^\dagger \\ \mathbf{p}_{11}^\dagger - x_1 \mathbf{p}_{13}^\dagger \\ y_2 \mathbf{p}_{23}^\dagger - \mathbf{p}_{22}^\dagger \\ \mathbf{p}_{21}^\dagger - x_2 \mathbf{p}_{23}^\dagger \end{bmatrix} \mathbf{R} = \mathbf{0}, \quad (7)$$

gde je $\mathbf{p}_{ij}$ j -ta kolona matrice $\mathbf{P}_i$. Skraćeno:

$$\mathbf{A} \mathbf{R} = \mathbf{0}. \quad (8)$$

U praktičnoj primeni, rešavanje jednakosti (8) biće jalovo — rešenje najčešće nećemo dobiti jer se zbog postojanja nenulte greške merenja zraci koji upadaju u kamere a na kojima leže merene tačke u najvećem broju slučajeva razmimoilaze. Problem, dakle, treba modifikovati u minimizaciju.

$$\mathbf{R} = \arg \min_{\mathbf{Q}, \|\mathbf{Q}\|=1} \|\mathbf{A} \mathbf{Q}\| \quad (9)$$

($\mathbf{R}$ i $\mathbf{Q}$ dati su homogenim koordinatama; ograničenje intenziteta jeste fiktivno i služi da omogući dalji rad). Primenjuje se jedan vrlo elegantan metod rešavanja (9). Naime, ukoliko je singularna dekompozicija matrice $\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\dagger$, garantuje se da je traženo $\mathbf{R}$ dato krajnjom desnom kolonom matrice $\mathbf{V}$.

Implementacija ovog algoritma u kodu data je ispod. Nema potrebe raditi konverziju u bilo koje konkretne jedinice dužine — ostatak posla zasnovan je na odnosima. Koordinatnim početkom smatra se ugao kartonskog nosača. Napomena: konvencija imenovanja je izmenjena!

```
def trijagulacijaTacke(r1, r2):
    kameraB = np.array([
        [1, 0, 0, -1],
        [0, 0, -1, 0],
        [0, 1, 0, 0]
    ])
    kameraB = np.dot(0.5*np.array([
        [772.81013182, 0, 320],
        [0, 779.95788961, 240 ],
        [0,0,2]
    ]), kameraB)
    kameraA = np.array([[0, -1, 0, 1],
        [0, 0, -1, 0],
        [1, 0, 0, 0]
    ])
    kameraA = np.dot(0.5*np.array([[768.3329645, 0, 320],
        [0, 770.36287522, 240 ],
        [0,0,2]
    ]), kameraA)
    A = np.array([
        (r1[1])*kameraA[2] - kameraA[1],
```

```

kameraA[0] - (r1[0])*kameraA[2],
(r2[1])*kameraB[2] - kameraB[1],
kameraB[0] - (r2[0])*kameraB[2],
])
U, S, Vt = np.linalg.svd(A)
trazeni = Vt[-1]
izlaz = trazeni[:3]/trazeni[-1]
return izlaz

```

UTVRĐIVANJE UPRAVLJAČKIH BROJEVA

Kako je preslikavanje $\Delta \mathbf{p} \rightarrow \Delta \mathbf{R}$ (gde je $\Delta \mathbf{R}$ vektor pomeraja od korena šake do vrha prsta, a $\Delta \mathbf{p}$ projekcija istog na pravac određen opruženim prstom) za robotsku šaku jednoznačno, upravljanje se svodi na praćenje odnosa $\frac{\|\Delta \mathbf{p}\|}{\|\Delta \mathbf{R}_0\|}$ za ljudsku šaku, gde je $\|\Delta \mathbf{R}_0\|$ dužina opruženog prsta, i evaluaciju inverza modela prsta nad tim odnosom (unutar zadatih granica kodomena).

U zavisnosti od metoda koji smo primenili za merenje značajnih koordinata razlikujemo i dva postupka merenja vektora opruženog prsta $\Delta \mathbf{R}_0$.

- (1) Ukoliko se koristi biblioteka *mediapipe*, pravac opruženog prsta može se aproksimirati vektorom pomeraja od korena šake do korena prsta (smatramo da opruženi prst leži u ravni dlana, na tom pravcu), a dužina opruženog prsta — zbirom dužina svih njegovih segmenata od korena šake do vrha prsta.

$$\Delta \mathbf{R}_0 := \sum_{prst} \|\Delta \mathbf{R}_{segment}\| \cdot \frac{\mathbf{R}_{koren\ prsta} - \mathbf{R}_{koren\ sake}}{\|\mathbf{R}_{koren\ prsta} - \mathbf{R}_{koren\ sake}\|} \quad (10)$$

- (2) Ako se pak koristi „domaći” pristup sa segmentacijom po boji, taj vektor mora se odrediti pomoću slika raširene šake, prostim oduzimanjem koordinata vrha prsta i referentne plave tačke.

$$\Delta \mathbf{R}_0 :=_{(ref.\ slike)} \mathbf{R}_{vrh\ prsta} - \mathbf{R}_{ref} \quad (11)$$

Vektor prsta $\Delta \mathbf{R}$ u oba slučaja određuje se oduzimanjem vrha prsta i ref. tačke (u slučaju prvog pristupa to je koren šake, a u slučaju drugog — plava tačka).

$$\Delta \mathbf{R} := \mathbf{R}_{vrh\ prsta} - \mathbf{R}_{ref} \quad (12)$$

Vektor $\Delta \mathbf{p}$ dat je sledećim obrascem:

$$\Delta \mathbf{p} := \|\Delta \mathbf{R}\| \cos \angle(\Delta \mathbf{R}_0, \Delta \mathbf{R}) \frac{\Delta \mathbf{R}_0}{\|\Delta \mathbf{R}_0\|} = \frac{\Delta \mathbf{R}_0 \cdot \Delta \mathbf{R}}{\Delta \mathbf{R}_0 \cdot \Delta \mathbf{R}_0} \Delta \mathbf{R}_0. \quad (13)$$

Traženi odnos očevitno jeste:

$$\frac{\|\Delta \mathbf{p}\|}{\|\Delta \mathbf{R}_0\|} = \frac{\Delta \mathbf{R}_0 \cdot \Delta \mathbf{R}}{\Delta \mathbf{R}_0 \cdot \Delta \mathbf{R}_0}. \quad (14)$$

Kretanje prsta modelovano je izvesnim preslikavanjem $f : \theta \rightarrow \frac{\|\Delta \mathbf{p}\|}{\|\Delta \mathbf{R}_0\|}$, gde je $\theta \in [0, 1]$ ugao otklona motora. Funkcija f je za svaki prst uzorkovana u 180

ravnomerno raspoređenih tačaka domena između 0 i 1 i zabeležena u lookup tabelu. Kako $\theta \sim N$, gde je $N \in [0, 180] \cap \mathbb{Z}$ broj zadat motoru — pri čemu $N = 180$ rezultuje maksimalnim savijanjem prsta, a $N = 0$ potpunim opružanjem — indeks željenog $\frac{\|\Delta \mathbf{P}\|}{\|\Delta \mathbf{R}_0\|}$ u tabeli jeste broj koji treba zadati motoru. Za to se koristi binarna pretraga, tj. funkcija *bisect* biblioteke *bisect*.

Kod za slučaj kada se koristi biblioteka *mediapipe* dat je ispod. Napomena: konvencija imenovanja je izmenjena; kod je malo modifikovan radi lakšeg razumevanja.

```
def utvrđiDuzinuOpruzenog(tT, rbr):
    out = 0
    if rbr in range(1, 5):
        out = np.linalg.norm(tT[0]-tT[1+rbr*4])
        +np.linalg.norm(tT[1+rbr*4]-tT[2+rbr*4])
        +np.linalg.norm(tT[2+rbr*4]-tT[3+rbr*4])
        +np.linalg.norm(tT[3+rbr*4]-tT[4+rbr*4])
    elif rbr==0:
        out = np.linalg.norm(tT[0] - tT[2])
        +np.linalg.norm(tT[2]-tT[3])
        +np.linalg.norm(tT[3]-tT[4])
    return out

def projekcija(tT, rbr):
    if rbr in range(1, 5):
        P = tT[1+4*rbr]-tT[0]
        R = tT[4+4*rbr]-tT[0]
        return (R@P)/(P@P)*P
    elif rbr==0:
        P = tT[2]-tT[0]
        R = tT[4]-tT[0]
        return (R@P)/(P@P)*P

def upravljackiBrojevi():
    global trijanulisaneTacke
    duzinePruzenih = np.array(
        [utvrđiDuzinuOpruzenog(trijangulisaneTacke, i) for i in range(5)]
    )
    duzineProjekcija = np.array(
        [np.linalg.norm(projekcija(trijangulisaneTacke, i)) for i in range(5)]
    )
    omer = np.zeros(5, dtype=np.float32)
    omer = duzineProjekcija/duzinePruzenih
    izlazi = np.zeros(5, dtype=np.uint8)
    izlazi[0] = bisect.bisect(-palacModel, -omer[0])
    izlazi[1] = bisect.bisect(-kaziprstModel, -omer[1])
    izlazi[2] = bisect.bisect(-srednjakModel, -omer[2])
    izlazi[3] = bisect.bisect(-domaliModel, -omer[3])
    izlazi[4] = bisect.bisect(-maliModel, -omer[4])
    return izlazi
```

Sledi primer koda (za mali prst) za slučaj kada se koristi segmentacija po boji. Napomena: konvencija imenovanja je izmenjena; kod je malo modifikovan radi lakšeg razumevanja.

```
def upravljackiBrojMali():
    global izlaziz1
    global izlaziz2
```

```

global izlazz1
global izlazz2
P = izlaziz1 - izlazir1
R = izlazz1 - izlazzr1
Pp = (R@P)/(P@P)
izlaz = bisect.bisect(-maliModel, -Mp)
return izlaz

```

Modeli prstiju dobijaju se fitovanjem parametrizovanih funkcija kroz merenja prstiju, na sledeći način:

```

def palacFja(x):
    return 0.56*np.sin(2.02*x+1.62)+0.449
def kaziprstFja(x):
    return 0.29*np.sin(2.89*x+1.87)+0.71
def srednjakFja(x):
    return 0.33*np.sin(2.46*x+1.94)+0.67
def domaliFja(x):
    return 0.38*np.sin(3.25*x+1.89)+0.64
def maliFja(x):
    return -0.45*x**2 -0.13*x + 0.99
palacModel = palacFja(np.linspace(0, 1, 180))
kaziprstModel = kaziprstFja(np.linspace(0, 0.9859, 176))
srednjakModel = srednjakFja(np.linspace(0, 1, 180))
domaliModel = domaliFja(np.linspace(0, 0.8684, 156))
maliModel = maliFja(np.linspace(0, 1, 180))

```

Prsti se u trenutku pisanja rada modeluju funkcijama svedenim na jedan harmonik i konstantu, tj. f-jama oblika $A \sin(B\theta + C) + D$, budući da se osmišljeni model „Kinderica” pokazao previše osetljivim na neidealnosti prstiju. Model malog prsta naknadno je razvijen po Tejloru do kvadratnog člana zbog uticaja nepreciznosti računa s pokretnim zarezo (dva parametra njegove sinusoide devetog su reda veličina, a ostali brojevi s kojima radimo — reda 10^{-1} ili 10^1). Domeni određenih f-ja suženi su zbog neočekivanog javljanja prevojnih tačaka na domenu $[0, 1]$ (tj. dvoznačnosti inverza).

KODOVANJE I SLANJE PORUKA

Upravljački brojevi koduju se u 128-bitnu poruku oblika

$$\overline{A_1 A_2 A_3 B_1 B_2 B_3 C_1 C_2 C_3 D_1 D_2 D_3 E_1 E_2 E_3 N},$$

tako da je K_i reprezentacija u kodu ASCII i -te cifre sleva trocifrene dekadne reprezentacije broja K , a N znak u kodu ASCII za novi red. Takvu poruku posebna nit prosleđuje sistemu ω primenom metoda *write* klase *Serial* biblioteke *pyserial*. Kod je prikazan ispod.

```

def arduinoIzlaz():
    arduino = serial.Serial(port='/dev/ttyACM0', baudrate=115200, timeout=.2)
    vreme = 0
    global komanda
    while True:
        arduino.write(bytes(komanda, 'ascii'))

```



```

    arduino.reset_output_buffer()
nit3 = threading.Thread(target = arduinoIzlaz, args=())
nit3.start()

while True:
    ...
    komanda = "".join([f"{x:03}" for x in izlazi]) + "\n"

```

SISTEM ω

Softver sistem ω krajnje je jednostavan i jedina mu je namena primljene upravljačke brojeve konvertovati u upravljačke signale za motore. Kada se u baferu USB porta nađe celovita poruka dužine 128 bita, Arduino Uno je prihvati, iscepa na troslovne segmente i odbaci znak za novi red, protumači segmente kao dekadne brojeve i na osnovu njih generiše odgovarajuće PWM signale na pinovima na koje su vezani motori. Kod je priložen ispod.

```

#include<Servo.h>
Servo servo[5];
int angle[5] = { 0 };
int motori[5] = {3, 5, 6, 9, 10};
int c = 0;
String ulaz;
String nizUlaza[5] = { "" };
void setup() {
    Serial.begin(115200);
    Serial.setTimeout(10);
    for (int i = 0; i < 5; i++)
    {
        servo[i].attach(motori[i]);
        servo[i].write(angle[i]);
    }
}
void loop() {
    while (Serial.available()>=16)
    {
        ulaz = Serial.readStringUntil('\n');
        for (int i = 0; i < 5; i++)
        {
            nizUlaza[i] = "";
            for (int j = 0; j < 3; j++)
                nizUlaza[i] += ulaz[3 * i + j];
        }
        for (int i = 0; i < 5; i++)
            angle[i] = nizUlaza[i].toInt();
    }
    for (int i = 0; i < 5; i++)
        servo[i].write(angle[i]);
}

```